

Contents

1	Introduction	2
2	Aim	2
3	Objectives	2
4	Scope	2
4.1	Core release	2
4.2	Future or out of scope	3
5	Users	3
6	System architecture	3
6.1	Current deployment model	3
7	Technology stack	4
8	Main interfaces	4
9	SEO and accessibility	4
10	Implementation method	4
11	Deployment workflow	5
12	Evaluation	5
13	Expected deliverables	5
14	Conclusion	6

1 Introduction

Python is used in software development, data science, automation, web development, and education. However, learning resources are often fragmented across many websites and do not always provide a clear progression from beginner concepts to practical projects.

Python Learning Hub will be a structured learning and community platform that combines a guided Python roadmap, searchable educational content, practical examples, learner progress, bookmarks, and discussions. Search-engine-friendly public pages will make the content easy to discover, while a Flutter Web application will support richer authenticated workflows.

2 Aim

The aim of this project is to design and implement a lightweight, SEO-optimized Python learning platform that provides structured education and demonstrates a maintainable modern web architecture.

3 Objectives

The project will:

1. provide a beginner-to-advanced Python learning roadmap;
2. publish clear topics, tutorials, guides, examples, categories, and tags;
3. support public search and related-content discovery;
4. provide authentication, progress tracking, bookmarks, and discussions;
5. expose a documented versioned API for interactive application features;
6. implement semantic HTML, clean URLs, metadata, canonical links, sitemap, and `robots.txt`;
7. provide a responsive and accessible user experience;
8. evaluate the system through functional, SEO, performance, and usability testing.

4 Scope

4.1 Core release

The first release will include approximately 15–25 Python topics, 5–10 tutorials or guides, and selected introductions to libraries and frameworks such as NumPy, Pandas, FastAPI, Django, asyncpg, and Requests. It will include:

- public learning pages and roadmap navigation;
- categories, tags, slugs, breadcrumbs, related content, and search;
- user registration and secure authentication;
- topic progress and bookmarks;
- public discussions with authenticated participation;
- basic administration and SEO metadata management.

4.2 Future or out of scope

Quizzes, advanced statistics, moderation dashboards, Redis caching, WebSockets, chat, media uploads, and expanded library coverage will be introduced incrementally. Payments, video hosting, live classes, native mobile applications, AI tutoring, online code execution, and guaranteed search rankings are outside the initial scope.

5 Users

User	Primary capabilities
Viewer	Browse and search public learning content without authentication.
Learner	Register, track progress, save bookmarks, and participate in discussions.
Administrator	Manage users, content, SEO metadata, discussions, reports, and statistics.

6 System architecture

FastAPI will be the single application boundary and will own the public website, API, and Flutter application. The platform will use two presentation layers:

- **Jinja2 and HTMX** for crawlable public pages and small progressive interactions;
- **Flutter Web** for authenticated dashboards and richer interactive workflows.

Both layers will use shared services and repositories. Neither frontend will access PostgreSQL directly.

```
Browser
|
v
FastAPI
|-- Jinja2 + HTMX public pages: /
|-- Versioned JSON API: /api/v1
|-- Flutter Web application: /app
'-- Health and documentation: /health, /ready, /docs
    |
    '-- PostgreSQL through asyncpg
```

6.1 Current deployment model

The Flutter source is kept in `frontend/`. The build helper creates a Web artifact with `/app/` as its base path and copies it into `backend/app/flutter/web`. FastAPI then serves the artifact at `/app`, including client-side routes such as `/app/dashboard`. The production application therefore uses one FastAPI origin:

```
https://seo-python-hub-437e0515.fastapicloud.dev/
https://seo-python-hub-437e0515.fastapicloud.dev/app
https://seo-python-hub-437e0515.fastapicloud.dev/api/v1
```

7 Technology stack

Area	Technology or approach
Backend	Python, FastAPI, Uvicorn, Jinja2, HTMX
Frontend	Flutter Web and semantic server-rendered HTML
Database	PostgreSQL with asynchronous <code>asyncpg</code> queries
Migrations	Alembic or versioned SQL migrations
Caching	Redis for selected read-heavy and temporary operations
API	Versioned JSON endpoints under <code>/api/v1</code>
Deployment	FastAPI Cloud with embedded Flutter Web artifact

The project will keep database access in repositories and business rules in services. Request and response schemas will define API contracts. The database remains the durable source of truth; Redis will not replace PostgreSQL.

8 Main interfaces

Public HTML routes will include:

GET /	Home page
GET /roadmap	Learning roadmap
GET /topics/{slug}	Topic page
GET /tutorials	Tutorial listing
GET /guides	Guide listing
GET /search	Search page
GET /discussions	Public discussions
GET /sitemap.xml	Published-page sitemap
GET /robots.txt	Crawler rules

The first API release will prioritize authentication, topics, roadmap, progress, bookmarks, search, discussions, and system health. The Flutter client will use the API through the same FastAPI origin in production.

9 SEO and accessibility

Public pages will use semantic HTML and server-side rendering. Each indexable page should provide a descriptive title, meta description, one primary heading, canonical URL, meaningful internal links, breadcrumbs where appropriate, and structured data only when accurate. The system will provide a sitemap, crawler rules, useful error pages, responsive layouts, and accessible markup.

10 Implementation method

Development will proceed in stages:

1. stabilize configuration, health endpoints, routing, tests, and deployment;
2. add PostgreSQL migrations, repositories, schemas, and services;
3. implement public content, search, categories, tags, and SEO metadata;
4. implement authentication, progress, bookmarks, and discussions;
5. connect Flutter screens to completed API contracts;

6. add administration, quizzes, analytics, Redis, and real-time features incrementally.

Every feature should be implemented through the shared service layer before adding separate Jinja2, HTMX, or Flutter representations.

11 Deployment workflow

From the repository root:

```
python scripts/build_flutter.py
cd backend
uv run pytest
uv run fastapi deploy .
```

The build helper runs Flutter dependency installation, builds with `-base-href /app/`, embeds the FastAPI API URL through `API_BASE_URL`, verifies the output, and copies it into the backend package. GitHub Actions validates the build and tests. FastAPI Cloud is responsible for the production deployment.

12 Evaluation

The project will be evaluated through:

- functional and integration tests for navigation, APIs, authentication, content, and learner actions;
- backend and Flutter automated tests;
- Lighthouse SEO and accessibility checks targeting at least 90/100;
- a target Lighthouse performance score of at least 85/100;
- usability testing with approximately 5–10 users;
- content review for accuracy, consistency, examples, links, and progression.

13 Expected deliverables

The completed project will deliver:

1. a structured Python learning website and roadmap;
2. public SEO-friendly content and search;
3. authentication, progress, bookmarks, and discussions;
4. a documented versioned FastAPI API;
5. a Flutter Web application served below `/app`;
6. PostgreSQL migrations, repositories, services, and tests;
7. deployment and maintenance documentation;
8. functional, SEO, performance, usability, and content-quality results.

14 Conclusion

Python Learning Hub will combine discoverable educational content with a richer authenticated learning experience. FastAPI will provide one reliable application boundary, Jinja2 and HTMX will preserve SEO-friendly public pages, and Flutter Web will support interactive workflows without creating a second production backend. The staged approach keeps the initial release achievable while leaving room for community, administration, and real-time capabilities.